

Rust en sistemas críticos: hacia un software más seguro y fiable

Ikram El Mabrouk Morhnane, UO301848 y Eva Menéndez Hernández, UO300181

En el desarrollo de software moderno hay un objetivo que cada vez pesa más que cualquier otro: la fiabilidad. No todos los programas tienen el mismo nivel de exigencia, pero hay una categoría en la que el margen de error es prácticamente inexistente: los sistemas críticos.

Este artículo nace a partir de una presentación sobre el uso de Rust en este tipo de sistemas, y pretende explicar de forma clara por qué este lenguaje está ganando tanta relevancia en contextos donde un fallo no es una opción.

Cuando el software no puede fallar

No todos los errores tienen el mismo impacto. En una aplicación común, un fallo puede suponer una mala experiencia de usuario o la pérdida de datos. Sin embargo, en un sistema crítico, las consecuencias pueden ser mucho más graves.

Se habla de sistemas *mission-critical* cuando un fallo provoca pérdidas económicas o interrupciones importantes del servicio. Por otro lado, los sistemas *safety-critical* van un paso más allá: un error puede poner en riesgo la seguridad de las personas o incluso sus vidas.

Este tipo de sistemas está mucho más presente en nuestra vida de lo que parece. Desde dispositivos médicos hasta software aeronáutico, pasando por sistemas de control de vehículos o infraestructuras tecnológicas, todos ellos dependen de que el software funcione correctamente en todo momento. En este contexto, la fiabilidad no es una característica deseable, sino una obligación.

Más que código: estándares y certificaciones

Desarrollar software para sistemas críticos implica cumplir con estándares muy estrictos. No basta con que el programa funcione; debe demostrar que lo hace de forma segura y predecible.

Existen normativas específicas según el sector, como ISO 26262 en automoción, IEC 61508 en entornos industriales o DO-178C en aviación. Estos estándares establecen procesos rigurosos que incluyen pruebas exhaustivas, validaciones formales, revisiones de código y auditorías externas.

Además, algo que a menudo pasa desapercibido es que no solo el software debe ser fiable. Las herramientas utilizadas para crearlo, como compiladores o toolchains, también deben estar certificadas. Un error en el compilador podría generar código incorrecto, incluso si el programa original es correcto.

Seguridad y ciberseguridad: dos mundos que convergen

Durante mucho tiempo, se ha diferenciado entre *safety* (seguridad física) y *security* (ciberseguridad). Sin embargo, esta separación es cada vez menos clara.

Hoy en día, muchos sistemas críticos están conectados a redes, e incluso a internet. Esto significa que un ataque informático ya no solo compromete datos, sino que puede tener consecuencias físicas reales. Un coche, un hospital o una infraestructura energética pueden verse afectados por vulnerabilidades en el software.

En este escenario, escribir código seguro ya no es solo una cuestión técnica, sino una necesidad fundamental.

Rust: un lenguaje diseñado para evitar errores

Es en este contexto donde aparece Rust. Se trata de un lenguaje de programación moderno, creado originalmente por Mozilla, cuyo objetivo es claro: permitir desarrollar software eficiente sin sacrificar la seguridad.

Rust está especialmente orientado a sistemas de bajo nivel, donde tradicionalmente se han utilizado lenguajes como C o C++. Estos lenguajes ofrecen un gran control y rendimiento, pero también son propensos a errores difíciles de detectar, especialmente relacionados con la memoria.

Aquí es donde Rust marca la diferencia. Su diseño permite evitar muchas de estas clases de errores antes incluso de que el programa se ejecute.

El problema de la memoria

Una gran parte de los fallos en software crítico tiene su origen en errores de memoria. Problemas como los *buffer overflows*, el uso de memoria ya liberada (*use-after-free*) o las condiciones de carrera (*data races*) son especialmente comunes en lenguajes que requieren gestión manual de memoria.

Estos errores no solo provocan fallos, sino que también pueden abrir la puerta a vulnerabilidades de seguridad.

Rust aborda este problema desde la base. En lugar de dejar la gestión de memoria completamente en manos del programador, introduce mecanismos que permiten al compilador verificar que el uso de la memoria es seguro. Y lo más importante: lo hace en tiempo de compilación.

Esto significa que muchos errores se detectan antes de ejecutar el programa, reduciendo drásticamente el riesgo en producción.

Ownership: una forma diferente de pensar

Para conseguir esta seguridad, Rust introduce un modelo diferente basado en conceptos como *ownership*, *borrowing* y *lifetimes*.

Aunque al principio pueden resultar complejos, estos conceptos permiten al compilador tener un control muy preciso sobre cómo se utilizan los datos en memoria. Cada valor tiene un único propietario, y cualquier acceso adicional debe hacerse de forma controlada.

Este enfoque elimina la necesidad de un recolector de basura y, al mismo tiempo, evita errores comunes en otros lenguajes. El resultado es un equilibrio poco habitual: seguridad sin sacrificar rendimiento.

Comparación con C y C++

Durante décadas, C y C++ han sido la base del desarrollo de sistemas críticos. Su eficiencia los hace ideales para este tipo de aplicaciones, pero su modelo de memoria introduce riesgos importantes.

Rust no pretende reemplazarlos completamente, pero sí ofrecer una alternativa más segura. Mantiene un rendimiento comparable, pero añade garantías adicionales gracias a su sistema de tipos y a las comprobaciones en compilación.

Además, Rust facilita algo especialmente complejo en otros lenguajes: la concurrencia segura. En sistemas donde múltiples procesos acceden a recursos compartidos, esto es fundamental.

De la teoría a la práctica

Rust ya está dando el salto al mundo real. Grandes empresas tecnológicas lo están incorporando en sus sistemas, especialmente en áreas donde la seguridad es crítica.

Se utiliza en sistemas embebidos, infraestructuras cloud, servidores y también en sectores como la automoción. Su adopción no es masiva todavía, pero la tendencia es clara: cada vez más organizaciones confían en Rust para reducir riesgos.

El papel de la industria: Florian Gilcher y Ferrocene

Una figura destacada en este ámbito es Florian Gilcher, ingeniero especializado en sistemas de bajo nivel, cofundador y CEO de Ferrous Systems, miembro de la Rust Foundation y uno de los impulsores del uso de Rust en entornos críticos.

A través de su empresa, Ferrous Systems, se ha desarrollado Ferrocene, una toolchain basada en Rust diseñada específicamente para cumplir con los requisitos de certificación industrial.

Ferrocene no solo proporciona un compilador, sino también una base formal del lenguaje, trazabilidad en los tests y validaciones en múltiples arquitecturas. Todo ello orientado a hacer posible el uso de Rust en sistemas que deben cumplir estándares estrictos.

Los retos de adopción

A pesar de sus ventajas, Rust no está exento de desafíos. Su curva de aprendizaje puede ser más pronunciada, especialmente para desarrolladores acostumbrados a otros lenguajes.

Además, en entornos críticos, cualquier cambio tecnológico implica riesgos. Por eso, la adopción de Rust suele ser progresiva. Muchas empresas empiezan con proyectos pequeños o integran Rust en partes concretas de sistemas existentes escritos en C o C++.

Este enfoque permite aprovechar sus beneficios sin comprometer la estabilidad de sistemas ya en producción.

Una mirada al futuro

Rust representa una evolución importante en la forma de desarrollar software seguro. Su capacidad para eliminar errores en tiempo de compilación lo convierte en una herramienta especialmente valiosa en sistemas críticos.

Sin embargo, su éxito no depende solo del lenguaje en sí, sino también del ecosistema que lo rodea: herramientas certificadas, procesos adecuados y una adopción responsable.

Todo indica que, en los próximos años, Rust jugará un papel cada vez más relevante en el desarrollo de software donde la seguridad no es opcional, sino imprescindible.

Sobre esta presentación

Esta entrada está basada en una presentación inspirada en el podcast sobre Rust en sistemas críticos “*SE Radio 690: Florian Gilcher on Rust for Safety-Critical Systems*” en **SE Radio** (Software Engineering Radio).